

SIMATS ENGINEERING

ASSESSMENT TOOL 7

COURSE NAME: Operating Systems

COURSE CODE: CSA-04

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements **Assessment Tool Weightage:** 40%

QUESTIONS: 5 **Total Marks:**50

Annexure A

EXPERIMENTS

Code of Conduct:

I Atchaya Chandran R(192521394) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in blue ink, reading "R. Atchaya", is written over a faint, circular official stamp. The signature is fluid and cursive.

Experiment Questions:

- 1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.
- 2.Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.
- 3.Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.
- 4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.
- 5.Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.

RUBRICS FOR CALCULATION:

Criteria	Weight age (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Criteria	Weight age (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

--	--	--	--	--	--

1. Logical Address to Physical Address using Base and Limit Registers

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int base, limit, logical, physical;
```

```
    printf("Enter Base Register value: ");
```

```
    scanf("%d", &base);
```

```
    printf("Enter Limit Register value: ");
```

```
    scanf("%d", &limit);
```

```
    printf("Enter Logical Address: ");
```

```
    scanf("%d", &logical);
```

```
if (logical >= 0 && logical < limit) {  
    physical = base + logical;  
  
    printf("\nAddress Translation\n");  
    printf("-----\n");  
    printf("Logical Address : %d\n", logical);  
    printf("Base Register   : %d\n", base);  
    printf("Limit Register    : %d\n", limit);  
    printf("Physical Address : %d\n", physical);  
    printf("Status           : Valid Address\n");  
} else {  
    printf("\nLogical Address : %d\n", logical);  
    printf("Status           : Invalid Address\n");  
    printf("Memory access denied.\n");  
}  
  
return 0;  
}
```

Output:

Output

```
Enter Base Register value: 1000
Enter Limit Register value: 500
Enter Logical Address: 250
```

Address Translation

```
-----
Logical Address   : 250
Base Register     : 1000
Limit Register    : 500
Physical Address  : 1250
Status           : Valid Address
```

2. Paging Memory Management

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    int pageSize, numPages, numFrames;
```

```
    int pageTable[50];
```

```
    int pageNumber, offset;
```

```
    int frameNumber, physicalAddress;
```

```
    int i;
```

```
    printf("Enter Page Size: ");
```

```
    scanf("%d", &pageSize);
```

```
    printf("Enter Number of Pages: ");
```

```
    scanf("%d", &numPages);
```

```

printf("Enter Number of Frames: ");

scanf("%d", &numFrames);


printf("\nEnter Page Table Entries\n");


for (i = 0; i < numPages; i++) {

    printf("Page %d -> Frame: ", i);

    scanf("%d", &pageTable[i]);

}


printf("\nEnter Page Number: ");

scanf("%d", &pageNumber);


printf("Enter Offset: ");

scanf("%d", &offset);


if (pageNumber < 0 || pageNumber >= numPages) {

    printf("\nInvalid Page Number.\n");

}

else if (offset < 0 || offset >= pageSize) {

    printf("\nInvalid Offset.\n");

}

else {

    frameNumber = pageTable[pageNumber];

    physicalAddress = frameNumber * pageSize + offset;


    printf("\nPaging Address Translation\n");

    printf("-----\n");

    printf("Page Number    : %d\n", pageNumber);

    printf("Offset        : %d\n", offset);

    printf("Frame Number   : %d\n", frameNumber);

```

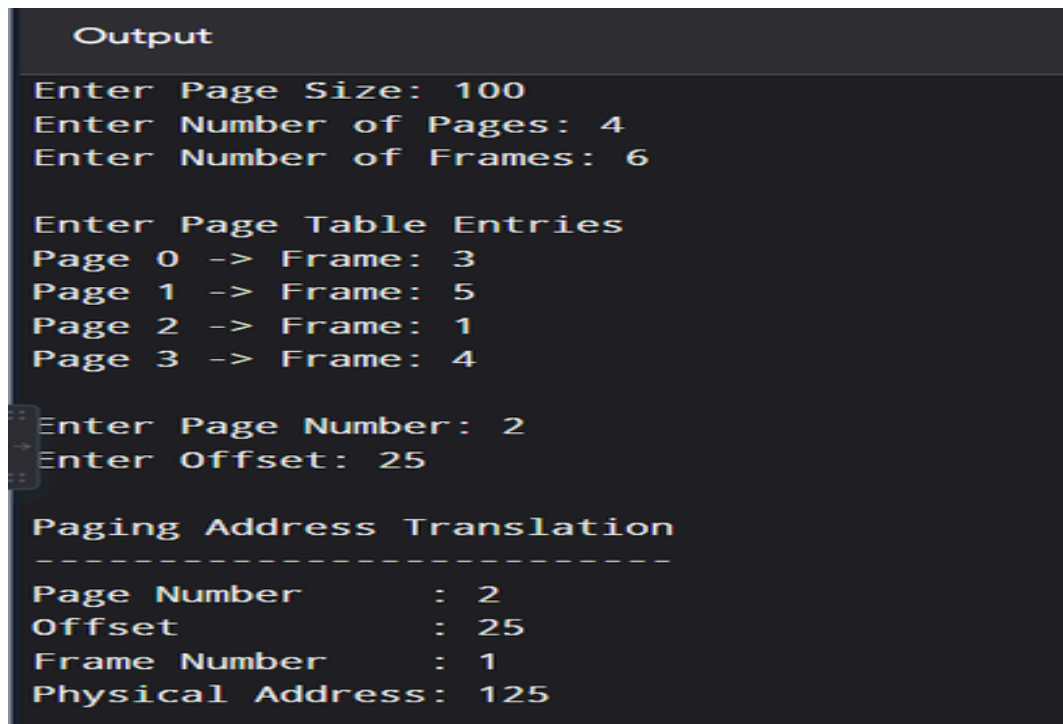
```

        printf("Physical Address: %d\n", physicalAddress);
    }

    return 0;
}

```

Output:



```

Output
Enter Page Size: 100
Enter Number of Pages: 4
Enter Number of Frames: 6

Enter Page Table Entries
Page 0 -> Frame: 3
Page 1 -> Frame: 5
Page 2 -> Frame: 1
Page 3 -> Frame: 4

Enter Page Number: 2
Enter Offset: 25

Paging Address Translation
-----
Page Number      : 2
Offset           : 25
Frame Number      : 1
Physical Address: 125

```

3. Page Table Simulation

Code:

```

#include <stdio.h>

int main() {
    int pageTable[50];
    int numPages;
    int pageSize;
    int logicalAddress;
    int pageNumber;

```



```
int offset;
```

```
int frameNumber;
```

```
int physicalAddress;
```

```
int i;
```

```
printf("Enter Page Size: ");
```

```
scanf("%d", &pageSize);
```

```
printf("Enter Number of Pages: ");
```

```
scanf("%d", &numPages);
```

```
printf("\nEnter Page Table\n");
```

```
printf("-----\n");
```

```
for (i = 0; i < numPages; i++) {
```

```
    printf("Page %d -> Frame: ", i);
```

```
    scanf("%d", &pageTable[i]);
```

```
}
```

```
printf("\nEnter Logical Address: ");
```

```
scanf("%d", &logicalAddress);
```

```
pageNumber = logicalAddress / pageSize;
```

```
offset = logicalAddress % pageSize;
```

```
if (pageNumber < 0 || pageNumber >= numPages) {
```

```
    printf("\nInvalid Logical Address.\n");
```

```
}
```

```
else {
```

```
    frameNumber = pageTable[pageNumber];
```

```
physicalAddress = frameNumber * pageSize + offset;
```

```
printf("\nPage Table Translation\n");
```

```
printf("-----\n");
```

```
printf("Logical Address : %d\n", logicalAddress);
```

```
printf("Page Number   : %d\n", pageNumber);
```

```
printf("Offset       : %d\n", offset);
```

```
printf("Frame Number  : %d\n", frameNumber);
```

```
printf("Physical Address: %d\n", physicalAddress);
```

```
}
```

```
return 0;
```

```
}
```

Output:

Output

```
Enter Page Size: 100
Enter Number of Pages: 5

Enter Page Table
-----
Page 0 -> Frame: 2
Page 1 -> Frame: 4
Page 2 -> Frame: 1
Page 3 -> Frame: 5
Page 4 -> Frame: 3

Enter Logical Address: 225

Page Table Translation
-----
Logical Address : 225
Page Number      : 2
Offset           : 25
Frame Number     : 1
Physical Address : 125
```

4. Translation Lookaside Buffer (TLB)

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    float memoryTime;
```

```
    float tlbTime;
```

```
    float hitRatio;
```

```
    float withoutTLB;
```

float withTLB;

printf("Enter Memory Access Time: ");

scanf("%f", &memoryTime);

printf("Enter TLB Access Time: ");

scanf("%f", &tlbTime);

printf("Enter TLB Hit Ratio (0 to 1): ");

scanf("%f", &hitRatio);

if (hitRatio < 0 || hitRatio > 1) {

 printf("\nInvalid Hit Ratio.\n");

 return 0;

}

withoutTLB = 2 * memoryTime;

withTLB = hitRatio * (tlbTime + memoryTime)

 + (1 - hitRatio) * (tlbTime + 2 * memoryTime);

printf("\nTLB Performance Analysis\n");

printf("-----\n");

printf("Memory Access Time : %.2f ns\n", memoryTime);

printf("TLB Access Time : %.2f ns\n", tlbTime);

printf("TLB Hit Ratio : %.2f\n", hitRatio);

printf("\nWithout TLB : %.2f ns\n", withoutTLB);

printf("With TLB : %.2f ns\n", withTLB);

if (withTLB < withoutTLB)

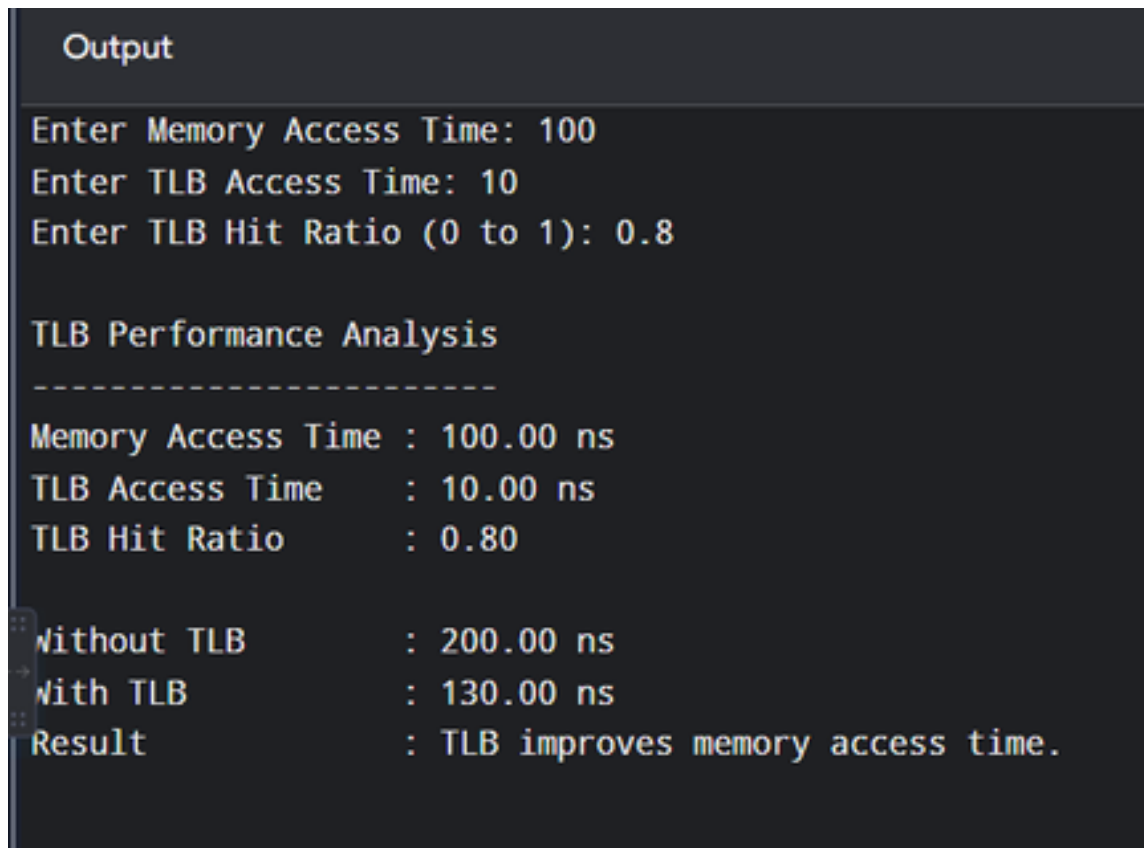
```

        printf("Result      : TLB improves memory access time.\n");
    else
        printf("Result      : TLB does not improve access time.\n");

    return 0;
}

```

Output:



```

Output

Enter Memory Access Time: 100
Enter TLB Access Time: 10
Enter TLB Hit Ratio (0 to 1): 0.8

TLB Performance Analysis
-----
Memory Access Time : 100.00 ns
TLB Access Time    : 10.00 ns
TLB Hit Ratio      : 0.80

Without TLB        : 200.00 ns
With TLB           : 130.00 ns
Result             : TLB improves memory access time.

```

5. Memory Protection using Base and Limit Registers

Code:

```
#include <stdio.h>
```

```
int main() {

    int base, limit, logical;

    int physical;


    printf("Enter Base Address: ");

    scanf("%d", &base);


    printf("Enter Limit: ");

    scanf("%d", &limit);


    printf("Enter Logical Address: ");

    scanf("%d", &logical);


    printf("\nMemory Protection Check\n");

    printf("-----\n");


    if (logical >= 0 && logical < limit) {

        physical = base + logical;


        printf("Logical Address : %d\n", logical);

        printf("Base Address   : %d\n", base);

        printf("Limit       : %d\n", limit);

        printf("Physical Address: %d\n", physical);

        printf("Access Status  : ALLOWED\n");

    }

    else {

        printf("Logical Address : %d\n", logical);

        printf("Access Status  : DENIED\n");

        printf("Reason       : Address is outside the valid range.\n");

    }

}
```

```
    return 0;  
}
```

Output:

```
Output  
Enter Base Address: 5000  
Enter Limit: 1000  
Enter Logical Address: 500  
  
Memory Protection Check  
-----  
Logical Address : 500  
Base Address    : 5000  
Limit           : 1000  
Physical Address: 5500  
Access Status   : ALLOWED
```